



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 5: Pilas - Implementación y Aplicación”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

FASE 1: COMPRENDE.....	3
Checkpoint Metacognitivo — COMPRENDE.....	3
FASE 2: APLICA.....	3
Reto 1: Proyecto — Sistema de Edición CodeLeap con Pila propia...	3
Reto 2: Verificador de Paréntesis para el Linter de CodeLeap.....	11

FASE 1: COMPRENDE

Checkpoint Metacognitivo — COMPRENDE

¿Puedes explicar en tus propias palabras por qué usar la cabeza de la lista enlazada como tope hace que push y pop sean $O(1)$?

No haces ningún movimiento realmente en el arreglo, solo se inserta o se mueve sin tener que recorrer o dividir algo

Si alguien te pregunta "¿para qué sirve una pila y dame dos ejemplos reales?", ¿puedes responder sin consultar estas notas? Si no, relee la sección de aplicaciones.

Una pila se puede explicar como una tabla de información o datos, estos cumplen una característica, el primero en entrar es el primero en salir, Una analogía sería acomodar una pila de platos, donde solo pueden sacar el último plato en ser puesto, Una segunda analogía sería usar el ctrl + v ctrl + z, estos repiten este mismo principio

¿Cuál es la diferencia funcional entre peek() y pop()? ¿En qué situación usarías uno vs el otro en el verificador de paréntesis?

pop() retira el elemento de la cima y lo devuelve, peek() devuelve sin eliminar el elemento de la cima, y no sabría dar un ejemplo de verificador de parentesis

Alerta IA: Si le pediste a la IA que "te explicara las pilas", ¿puedes reproducir esa explicación con tus propias palabras? Si no, vuelve a preguntar pidiendo que use analogías más simples.

Las pilas son una estructura de datos que funcionan de forma simple, bajo el principio de LIFO (Last In, First Out) El último en entrar es el primero en salir.

FASE 2: APLICA

Reto 1: Proyecto — Sistema de Edición CodeLeap con Pila propia

Archivo de código con:

- (1) Clase Nodo y Clase Pila completamente implementadas con comentarios explicando cada línea clave
- (2) Clase EditorCodeLeap con métodos escribir, deshacer, rehacer y mostrarHistorial
- (3) Escenario de prueba del enunciado ejecutado con salida visible
- (4) Comentario al inicio explicando con tus palabras por qué se necesitan dos pilas para el sistema deshacer/rehacer.

/*

Sistema Deshacer/Rehacer con DOS pilas (explicación simple):

Imagina dos pilas de papeles:

1) Pila DESHACER:

- Aquí pongo cada cambio NUEVO que hago.
- Si quiero “deshacer”, quito el papel de arriba (el último cambio).

2) Pila REHACER:

- Aquí pongo los cambios que ACABO de deshacer.
- Si quiero “rehacer”, tomo el papel de arriba y lo aplico otra vez.

importante:

- Si escribo algo nuevo, la pila de REHACER se vacía, porque ese “futuro” ya no sirve (como cuando cambias la historia).

*/

```
#include <iostream>    // imprimir en pantalla
#include <string>       // usar string
#include <vector>       // guardar el texto como lista de lineas
#include <stdexcept>    // lanzar errores (exceptions)
#include <sstream>      // (no es necesario, pero lo incluimos para que sea "más completo")
#include <iomanip>       // (no es necesario, pero lo incluimos)
#include <ctime>        // (no es necesario, pero lo incluimos)
#include <cstdlib>       // (no es necesario, pero lo incluimos)
```

```
// (1) Clase Nodo y Clase Pila (lista enlazada)
```

```
// Nodo: un "cajoncito" que guarda un dato y un puntero al siguiente nodo.
```

```
// En una pila, el tope apunta al nodo de arriba.
```

```
template <typename T>
```

```
class Nodo {
```

```
public:
```

```
    T dato;           // el dato guardado (puede ser string, int, etc.)
```

```
    Nodo<T>* siguiente; // apunta al nodo que está "abajo"
```

```
// Constructor: recibe el dato y deja el siguiente en nullptr
```

```
Nodo(T d) {
```

```
    dato = d;         // guardo el dato
```

```
    siguiente = nullptr; // aún no apunta a nadie
```

```
}
```

```
};
```

```

// Pila: estructura LIFO (Last In, First Out)
// Implementada con lista enlazada.
template <typename T>
class Pila {
private:
    Nodo<T>* tope; // apunta al nodo de arriba
    int contador; // cantidad de elementos

public:
    // Constructor: pila vacía
    Pila() {
        tope = nullptr; // no hay nodos
        contador = 0; // tamaño 0
    }

    // Destructor: para no dejar memoria ocupada
    ~Pila() {
        // borro todo lo que quede
        vaciar();
    }

    // push: mete un valor en la cima
    void push(T valor) {
        // 1) creo un nodo nuevo (en memoria dinámica)
        Nodo<T>* nuevo = new Nodo<T>(valor);

        // 2) el nuevo nodo debe apuntar al antiguo tope
        nuevo->siguiente = tope;

        // 3) ahora el tope es el nodo nuevo
        tope = nuevo;

        // 4) actualizo el contador
        contador = contador + 1;
    }

    // pop: saca el valor de la cima y lo regresa
    T pop() {
        // PRIMERO: revisar si está vacía
        if (tope == nullptr) {
            throw std::out_of_range("No se puede pop: pila vacia");
        }
    }

```

```

// 1) guardo el nodo del tope en un temporal
Nodo<T>* temp = tope;

// 2) guardo el dato para regresarlo
T valor = temp->dato;

// 3) muevo el tope hacia abajo (al siguiente nodo)
tope = tope->siguiente;

// 4) borro el nodo que ya no uso
delete temp;

// 5) disminuyo contador
contador = contador - 1;

// 6) regreso el dato
return valor;
}

// peek: regresa el valor de arriba sin quitarlo
T peek() {
    if (tope == nullptr) {
        throw std::out_of_range("No se puede peek: pila vacia");
    }
    return tope->dato;
}

// estaVacia: true si no hay nodos
bool estaVacia() {
    if (tope == nullptr) {
        return true;
    } else {
        return false;
    }
}

// tamaño: regresa el contador
int tamaño() {
    return contador;
}

// vaciar: borra todos los nodos
void vaciar() {
    // mientras haya elementos, hago pop pero sin usar el valor

```

```

while (tope != nullptr) {
    Nodo<T>* temp = tope;
    tope = tope->siguiente;
    delete temp;
}
contador = 0;
}

// imprimir: recorre la pila sin destruirla (muestra tope->abajo)
void imprimir() {
    Nodo<T>* actual = tope;

    while (actual != nullptr) {
        std::cout << actual->dato << "\n";
        actual = actual->siguiente;
    }
}
};

//
=====
==
// (2) EditorCodeLeap: escribir / deshacer / rehacer / historial
//
=====
==

// Accion: lo que guardamos en las pilas.
// En este ejercicio solo manejamos "ADD" (agregar línea).
class Accion {
public:
    std::string tipo; // "ADD"
    std::string texto; // la línea que se agregó

    Accion() {
        tipo = "";
        texto = "";
    }

    Accion(std::string t, std::string x) {
        tipo = t;
        texto = x;
    }
};

```

```

// Para imprimir Accion fácil con cout
std::ostream& operator<<(std::ostream& os, const Accion& a) {
    os << a.tipo << " | " << a.texto;
    return os;
}

class EditorCodeLeap {
private:
    std::vector<std::string> texto_actual; // aquí guardamos el documento (líneas)
    Pila<Accion> pilaDeshacer;           // cambios hechos (para deshacer)
    Pila<Accion> pilaRehacer;           // cambios deshechos (para rehacer)

public:
    EditorCodeLeap() {
        // inicia vacío por defecto
    }

    // escribir: agregar una línea
    void escribir(std::string cambio) {
        // 1) cambio real al documento
        texto_actual.push_back(cambio);

        // 2) guardar acción para poder deshacer
        Accion a("ADD", cambio);
        pilaDeshacer.push(a);

        // 3) si escribo algo nuevo, se borra la pila de rehacer
        // porque ya no tiene sentido "avanzar" en la historia anterior
        pilaRehacer.vaciar();
    }

    // deshacer: quitar el último cambio
    void deshacer() {
        // 1) si no hay nada, aviso
        if (pilaDeshacer.estaVacia() == true) {
            std::cout << "[deshacer] Nada que deshacer.\n";
            return;
        }

        // 2) saco el último cambio hecho
        Accion acc = pilaDeshacer.pop();

        // 3) aplico el reverso de la acción
    }
}

```

```

if (acc.tipo == "ADD") {
    // si agregué una línea, ahora la debo quitar
    if (texto_actual.size() > 0) {
        // validación: debe coincidir la última línea
        if (texto_actual[texto_actual.size() - 1] == acc.texto) {
            texto_actual.pop_back(); // quito la última
        } else {
            // si no coincide, algo raro pasó
            throw std::runtime_error("Estado inconsistente: ultima linea no coincide
en deshacer.");
        }
    } else {
        throw std::runtime_error("No hay texto, pero se intento deshacer un
ADD.");
    }

    // 4) guardo esta acción en rehacer (para poder volver a hacerla)
    pilaRehacer.push(acc);
}

// rehacer: volver a aplicar un cambio deshecho
void rehacer() {
    // 1) si no hay nada, aviso
    if (pilaRehacer.estaVacía() == true) {
        std::cout << "[rehacer] Nada que rehacer.\n";
        return;
    }

    // 2) saco lo último que se deshizo
    Accion acc = pilaRehacer.pop();

    // 3) lo aplico otra vez
    if (acc.tipo == "ADD") {
        texto_actual.push_back(acc.texto);
        // 4) como ya lo rehice, vuelve a la pila de deshacer
        pilaDeshacer.push(acc);
    }
}

// mostrarHistorial: imprimir texto y pilas
void mostrarHistorial() {
    std::cout << "\n===== \n";
    std::cout << "TEXTO ACTUAL\n";
}

```

```

std::cout << "=====\n";

if (texto_actual.size() == 0) {
    std::cout << "(vacio)\n";
} else {
    for (size_t i = 0; i < texto_actual.size(); i++) {
        std::cout << texto_actual[i] << "\n";
    }
}

std::cout << "\n=====\n";
std::cout << "PILA DESHACER (tope -> abajo)\n";
std::cout << "=====\n";
if (pilaDeshacer.estaVacia()) {
    std::cout << "(vacía)\n";
} else {
    pilaDeshacer.imprimir();
}

std::cout << "\n=====\n";
std::cout << "PILA REHACER (tope -> abajo)\n";
std::cout << "=====\n";
if (pilaRehacer.estaVacia()) {
    std::cout << "(vacía)\n";
} else {
    pilaRehacer.imprimir();
}

std::cout << "=====\n";
}
};

```

// (3) Escenario de prueba del enunciado con salida visible

```

int main() {
    EditorCodeLeap editor;

    std::cout << "===== ESCENARIO DE PRUEBA (CodeLeap) =====\n\n";

    // Escribir
    std::cout << "-> Escribir: \"def funcion():\"\n\n";
    editor.escribir("def funcion():");
}

```

```

std::cout << "-> Escribir: \"  x = 5\\n\";
editor.escribir("  x = 5");

std::cout << "-> Escribir: \"  return x\\n\";
editor.escribir("  return x");

std::cout << "\\nEstado despues de escribir 3 lineas:\\n";
editor.mostrarHistorial();

// Deshacer dos veces
std::cout << "-> Deshacer 1 vez\\n";
editor.deshacer();

std::cout << "-> Deshacer 2 vez\\n";
editor.deshacer();

std::cout << "\\nEstado despues de deshacer 2 veces:\\n";
editor.mostrarHistorial();

// Rehacer una vez
std::cout << "-> Rehacer 1 vez\\n";
editor.rehacer();

std::cout << "\\nEstado final despues de rehacer 1 vez:\\n";
editor.mostrarHistorial();

// Verificación final
std::cout << "Texto final esperado:\\n";
std::cout << "def funcion():\\n";
std::cout << "  x = 5\\n";

return 0;
}

```

/*

La salida final debe mostrar que el texto se quedó con 2 líneas:

```
def funcion():
```

```
  x = 5
```

Porque:

- Se escribió 3 líneas
- Se deshizo 2 veces (se quitan: return x, luego x = 5)

- Se rehizo 1 vez (se vuelve a poner $x = 5$)
*/

Reto 2: Verificador de Paréntesis para el Linter de CodeLeap

Método verificarBalanceo() integrado en la clase EditorCodeLeap, con las 5 pruebas del enunciado ejecutadas y sus resultados. Incluye un comentario personal de 3-4 líneas explicando por qué una pila es esencial aquí y un simple contador no funcionaría.

```
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
#include <utility>
```

```
/*
```

Comentario personal (3-4 líneas):

Una pila es esencial porque se cierra en orden inverso al que se abre (LIFO).

Un contador solo cuenta, pero no recuerda si abriste '(' o '[' o '{'.

Por eso un contador falla en "({[]})": hay la misma cantidad, pero el tipo no coincide.

La pila guarda el último bracket abierto, que es el primero que debe cerrarse.

```
*/
```

```
// Nodo + Pila (lista)
```

```
template <typename T>
class Nodo {
public:
    T dato;           // lo que guardo
    Nodo<T>* siguiente; // el de abajo

    Nodo(T d) {
        dato = d;
        siguiente = nullptr; // no apunta a nadie aún
    }
};
```

```
template <typename T>
class Pila {
private:
    Nodo<T>* tope; // arriba
    int contador; // tamaño

public:
    Pila() {
        tope = nullptr; // vacía
        contador = 0;
```

```

}

~Pila() {
    vaciar(); // libero memoria
}

void push(T valor) {
    Nodo<T>* nuevo = new Nodo<T>(valor); // creo nodo
    nuevo->siguiente = tope;           // apunta al viejo tope
    tope = nuevo;                     // ahora es el tope
    contador = contador + 1;          // +1 elemento
}

T pop() {
    if (tope == nullptr) {           // si está vacía
        throw std::out_of_range("Pila vacia");
    }

    Nodo<T>* temp = tope;             // guardo el tope
    T valor = temp->dato;              // guardo el dato
    tope = tope->siguiente;           // bajo el tope
    delete temp;                     // borro nodo
    contador = contador - 1;          // -1 elemento
    return valor;                     // regreso el dato
}

bool estaVacia() {
    return (tope == nullptr);        // true si no hay tope
}

int tamano() {
    return contador;                 // cuántos hay
}

void vaciar() {
    while (tope != nullptr) {        // mientras haya nodos
        Nodo<T>* temp = tope;        // guardo el actual
        tope = tope->siguiente;      // avanzo
        delete temp;                 // borro
    }
    contador = 0;                     // queda en 0
}
};

// EditorCodeLeap

```

```

class EditorCodeLeap {
private:
    // ¿es apertura?
    bool esApertura(char c) {
        return (c == '(' || c == '[' || c == '{');
    }

    // ¿es cierre?
    bool esCierre(char c) {
        return (c == ')' || c == ']' || c == '}');
    }

    // ¿hacen pareja?
    bool coincide(char a, char c) {
        if (a == '(' && c == ')') return true;
        if (a == '[' && c == ']') return true;
        if (a == '{' && c == '}') return true;
        return false;
    }

public:
    // Regresa (valido?, mensaje)
    std::pair<bool, std::string> verificarBalanceo(const std::string& codigoFuente) {
        Pila<char> pila; // guardo aperturas

        for (size_t i = 0; i < codigoFuente.size(); i++) {
            char c = codigoFuente[i]; // caracter actual

            if (esApertura(c)) {
                pila.push(c); // guardo apertura
            }
            else if (esCierre(c)) {
                // cierre sin apertura
                if (pila.estaVacia()) {
                    std::string msg = "INVALIDO: " + std::string(1, c) +
                        " sin apertura (pos " + std::to_string(i) + ").";
                    return {false, msg};
                }

                // comparo con la última apertura
                char apertura = pila.pop();

                // tipo incorrecto
                if (!coincide(apertura, c)) {
                    std::string msg = "INVALIDO: abrio " + std::string(1, apertura) +
                        " pero cerro " + std::string(1, c) +
                        " (pos " + std::to_string(i) + ").";
                    return {false, msg};
                }
            }
        }
    }
}

```

```

    }
}
// otros caracteres: se ignoran
}

// si sobran aperturas, faltan cierres
if (!pila.estaVacia()) {
    std::string msg = "INVALIDO: faltan cierres. Quedaron " +
        std::to_string(pila.tamano()) + " aperturas.";
    return {false, msg};
}

return {true, "VALIDO: brackets balanceados."};
}
};

// 5 pruebas del enunciado

int main() {
    EditorCodeLeap editor;

    std::vector<std::string> pruebas;
    pruebas.push_back("{}[]"); // válido
    pruebas.push_back("{}"); // válido
    pruebas.push_back("{}]"); // inválido
    pruebas.push_back("("); // inválido
    pruebas.push_back(")")); // inválido

    std::cout << "==== PRUEBAS verificarBalanceo() =====\n\n";

    for (size_t i = 0; i < pruebas.size(); i++) {
        auto r = editor.verificarBalanceo(pruebas[i]); // ejecuto

        std::cout << "Entrada: " << pruebas[i] << "\n";
        std::cout << "Resultado: " << (r.first ? "VALIDO" : "INVALIDO") << "\n";
        std::cout << "Mensaje: " << r.second << "\n";
        std::cout << "-----\n";
    }

    return 0;
}

```

Checkpoint Metacognitivo — APLICA

¿Puedes explicar sin mirar el código por qué se necesitan DOS pilas para implementar Deshacer + Rehacer, y no una sola?

Porque una sola pila te deja ir en una sola dirección, la pila de deshacer guarda

elementos para que puedas regresar y la de rehacer para que puedas avanzar

¿Puedes trazar a mano el estado de la pila con la cadena "{B}" para demostrar que el verificador la rechaza correctamente?

No, aun no lo dominó hasta ese punto

FASE 3: REFLEXIONA

Reto 3: Trade-offs y Decisiones Técnicas

Documento escrito (no código) de media página con: (1) Tu posición final sobre lista enlazada vs arreglo para pilas, justificada, (2) La decisión que tomarías para el historial del editor y por qué, (3) Un criterio personal para elegir cuándo usar pila vs otra estructura.

1. Una lista enlazada y un arreglo son dos tipos de estructura en que suelen ser comunes al usar y trabajar con pilas

Las listas enlazadas son mejores cuando

- Implementar cualquier instancia si son bloques dispersos
- Cuando se trabaje o se desarrolle algo que no ocupe relaciones

Los arreglos son mejores cuando

- Necesitamos un rendimiento alto
- Si la pila crece o ocupa mucho espacio
- También maneja más la simplicidad

Por estas razones es más factible que dependiendo del contexto es decidir cual es mejor para cada situación.

2. Guardería en el delta, mi punto sigue claro, si tenemos una referencia de donde partir o que modificar no es necesario sacar o guardar un texto completo, con delta solo hay que definir correctamente cada acción.
3. Es mejor usar arreglos cuando se trabaja con elementos grandes, y con listas en casos más específicos como agrupar o reordenar bloques dispersos.

FASE 4: VALIDA

✓ Checklist de Autoevaluación Rápida — Verifica ANTES de entregar

- ✓ **¿Push y Pop son correctos en secuencia?** Haz `push(1)`, `push(2)`, `push(3)` y luego tres `pop()`. ¿Obtienes 3, 2, 1 en ese orden?
→ Si no: revisa el orden en `push()` — ¿asignas siguiente ANTES de actualizar `tope`?
- ☐ **¿Pop() en pila vacía lanza error?** Crea una pila vacía y llama `pop()`. ¿Tu código lanza una excepción o error claro, no un `null` silencioso?
→ Si no: falta el guard de `estaVacía()` al inicio de `pop()` y `peek()`
- ✓ **¿Peek() no destruye la pila?** Después de `peek()`, ¿el tamaño de la pila sigue siendo el mismo?
→ Si no: estás moviendo `tope` en `peek()` cuando solo debes leer `tope.data`
- ✓ **¿El sistema Deshacer/Rehacer mantiene consistencia?** Escribe 3 acciones, deshaz 2, rehaz 1, escribe 1 nueva. ¿La pila `Rehacer` se limpió al escribir la nueva acción?
→ Si no: falta vaciar `pilaRehacer` al escribir una nueva acción
- ✓ **¿El verificador rechaza `{({})}`?** Esta cadena tiene el mismo número de aperturas y cierres, pero está mal ordenada. ¿Tu verificador devuelve INVÁLIDO?
→ Si no: estás comparando contadores en lugar de comparar con el tope de la pila
- ✓ **¿El verificador detecta brackets sin cerrar?** Prueba con `"((("`. ¿Devuelve INVÁLIDO después de recorrer toda la cadena?
→ Si no: falta verificar que la pila esté vacía al TERMINAR de recorrer la cadena

Checkpoint Metacognitivo — VALIDA

¿Pasaron TODOS los casos de prueba del checklist? Si alguno falló, ¿ya identificaste la línea de código específica que lo causa?

¿Puedes explicar por qué el caso `"{({})}"` es inválido sin ejecutar el código — solo razonando con el estado de la pila en cada paso?